DOCUMENTO OFICIAL DE INGENIERÍA & PEDAGOGÍA DUAL

Medicent Backend V3

Guía Maestra de Desarrollo: Explicación Intuitiva (5 Años) & Especificación Técnica

EQUIPO DE DESARROLLO
Cristian Beltrán & Compañero

STACK PRINCIPAL
Python 3.14 · Flask 3.1 · MySQL / SQLite

VERSIÓN / FECHA
v3.0 Oficial · Octubre 2026

1. ¿Qué es Medicent? Explicado para un Niño de 5 Años 🧸

🏥 La Historia del Hospital de Juguetes

Imagina que tenemos un hospital mágico para cuidar ositos y muñecos. En este hospital hay dos clases de amigos:

- **Los Pacientes (Los Ositos):** Son los muñequitos que se sienten cansados o enfermos y necesitan tomar una cucharadita de jarabe para sanar.
- **Los Cuidadores (Los Doctores Cariñosos):** Son los hermanos mayores que se aseguran de que ningún osito olvide tomar su medicina a la hora exacta.

El Gran Botiquín Ordenado (La Base de Datos): En el hospital no podemos tirar las pastillas por el suelo porque se perderían. Por eso tenemos una estantería mágica con 5 cajones de colores que nunca se desordenan:

- **Cajón 1 (Usuarios):** La lista con la foto y nombre de todos los ositos y doctores.
- **Cajón 2 (Pacientes y Cuidadores):** El lazo de amistad que dice qué doctor cuida a qué osito.
- **Cajón 3 (Medicamentos):** Las botellitas de jarabe, gotas y pomadas que existen en la farmacia.
- **Cajón 4 (Tratamientos):** La receta del doctor que dice: "*Toma 1 pastilla cada 8 horas por 5 días*".
- **Cajón 5 (Registro de Tomas):** Las alarmas mágicas que van sonando: "*Toma de las 8:00 AM: ¡Tomada!*", "*Toma de las 4:00 PM: ¡Pendiente!*".

El Candado y la Pulsera Mágica (Seguridad y Tokens JWT): Para que ningún extraño travieso entre a cambiar las medicinas, cada doctor tiene una palabra secreta. Al entrar al hospital, el guardián revisa su palabra y le coloca una pulsera brillante (un Token JWT). Si tienes la pulsera de Cuidador, las puertas del hospital se abren automáticamente para ti.

El Reloj Despertador Automático (Motor de Tomas): El doctor no tiene que poner 30 alarmas a mano en su teléfono. La computadora es un robot súper listo que suma las horas solita: 8:00 AM... luego 4:00 PM... luego medianoche... ¡y crea todas las alarmas en un segundo!

2. Especificación Técnica de la Base de Datos (`medicent_v3`) 🗄️

> 📐 **Normalización, Integridad y Diagrama Relacional**
>
> Toda la base de datos se encuentra normalizada en Tercera Forma Normal (3FN), traducida rigurosamente al idioma español y diseñada bajo el motor InnoDB de MySQL con soporte UTF-8 completo (`utf8mb4_unicode_ci`).

Diccionario de Datos Detallado

Tabla	Campo	Tipo de Dato	Restricciones	Propósito Arquitectónico
usuarios	id	INT	PRIMARY KEY, AUTO_INCREMENT	Identificador único inmutable del usuario.
	nombre	VARCHAR(100)	NOT NULL	Nombre completo de la persona.
	correo	VARCHAR(120)	NOT NULL, UNIQUE	Credencial de login única en todo el sistema.
	contrasena_hash	VARCHAR(255)	NOT NULL	Hash seguro generado mediante Flask-Bcrypt (Blowfish).
	rol	ENUM	'paciente', 'cuidador', 'admin'	Control de acceso basado en roles (RBAC).
pacientes_cuidadores	cuidador_id	INT	FK -> usuarios(id), ON DELETE CASCADE	Tabla intermedia de cardinalidad N:M. Clave primaria compuesta `(cuidador_id, paciente_id)` para evitar duplicidad de asignaciones.
	paciente_id	INT	FK -> usuarios(id), ON DELETE CASCADE	
	creado_en	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	
medicamentos	id	INT	PRIMARY KEY, AUTO_INCREMENT	Identificador del fármaco.
	nombre	VARCHAR(150)	NOT NULL	Nombre comercial o genérico del medicamento.
	descripcion	VARCHAR(255)	NULL	Indicación o resumen farmacológico.
	forma_farmaceutica	VARCHAR(100)	NOT NULL	Presentación: comprimido, suspensión, cápsula, inyectable.
tratamientos	id	INT	PRIMARY KEY, AUTO_INCREMENT	Identificador único de la prescripción.
	paciente_id	INT	FK -> usuarios(id)	Paciente al que pertenece el tratamiento.
	medicamento_id	INT	FK -> medicamentos(id)	Fármaco prescrito.
	dosis	VARCHAR(100)	NOT NULL	Cantidad prescrita (ej. "500 mg", "1 tableta").
	frecuencia_horas	INT	NOT NULL (CHECK > 0)	Intervalo entre tomas en horas (ej. 8, 12, 24).
	fecha_inicio / fin	DATE	NOT NULL	Rango temporal de validez del tratamiento.

Tabla	Campo	Tipo de Dato	Restricciones	Propósito Arquitectónico
	hora_inicial	TIME	NOT NULL	Hora de arranque de la primera toma del primer día.
registros_tomas	id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Identificador de alta escala para auditoría.
	tratamiento_id	INT	FK -> tratamientos(id), ON DELETE CASCADE	Enlace directo a la prescripción médica origen.
	fecha_hora_programada	DATETIME	NOT NULL	Timestamp exacto calculado por el motor.
	fecha_hora_toma	DATETIME	NULL	Timestamp real en el que el paciente tomó la dosis.
	estado	ENUM	'pendiente', 'tomada', 'omitida', 'retrasada'	Máquina de estados para seguimiento clínico.

3. Arquitectura Modular por Capas del Backend 🏛️

> 🍕 **Como un Restaurante de Cocineros Expertos**
>
> Para no hacernos bolas, el código del backend está dividido en puestos de trabajo donde cada uno tiene una única misión:
>
> - **Las Rutas (`routes`):** Son los camareros que reciben a los clientes en la puerta y leen la carta.
> - **Los Controladores (`controllers`):** Son los meseros jefes que revisan que el pedido esté bien escrito y no falten ingredientes.
> - **Los Servicios (`services`):** Son los chefs que cocinan la comida rica (calculan horarios, revisan recetas, aplican la matemática).
> - **Los Modelos (`models`):** Son los frascos y cajas donde se guardan los ingredientes limpiecitos.
> - **Los Filtros (`middlewares`):** Son los guardias de seguridad que no dejan entrar a nadie sin gafete.

Especificación Técnica de Capas

Capa / Directorio	Responsabilidad Técnica	Reglas de Dependencia
`app/config/`	Gestiona parámetros de entorno con `settings.py`: clase `Config` (MySQL para producción/desarrollo) y clase `TestConfig` (SQLite en memoria `sqlite:///:memory:` para suite TDD).	No depende de capas de aplicación. Solo de variables de sistema o `.env`.
`app/database/`	Inicializa las extensiones SQLAlchemy, Bcrypt, JWTManager y Migrate con el patrón desacoplado (sin vincular de inmediato a la app).	Exporta objetos `db`, `bcrypt`, `jwt`.
`app/models/`	Declara entidades ORM mapeadas a las tablas de la base de datos en español. Métodos de serialización `to_dict()` para respuestas JSON.	Depende de `db`. No conoce controladores ni rutas.
`app/services/`	Capa de lógica de negocio pura. Contiene algoritmos de cálculo de horarios, validaciones de correo duplicado, control de transiciones de estados de tomas.	Independiente de Flask HTTP (no recibe `request`). Retorna datos o arroja excepciones de dominio.
`app/controllers/`	Orquestadores HTTP. Extraen payloads (`request.get_json()`), llaman a la capa de servicios, atrapan errores y formatean respuestas con códigos de estado HTTP estándar.	Depende de los servicios y de los utilitarios de respuesta.
`app/routes/`	Define `Blueprint` de Flask y mapea URIs con verbos HTTP (GET, POST, PATCH). Aplica middlewares de autorización.	Solo asocia endpoints con funciones de controladores.
`app/middlewares/`	Seguridad perimetral. Decorador `@rol_requerido(*roles)` que verifica validez del JWT y valida que el claim `rol` tenga permiso de acceso.	Intercepta peticiones entrantes antes de llegar a la ruta.

Patrón Application Factory (`app/__init__.py`)

El núcleo de la aplicación utiliza el patrón `create_app(config_class)` para permitir inicializar instancias independientes tanto para el servidor Flask de producción como para el ejecutor de pruebas automatizadas TDD:

```
# app/__init__.py (Esquema conceptual)
def create_app(config_class="app.config.settings.Config"):
    app = Flask(__name__)
    app.config.from_object(config_class)

    # Inicializar extensiones
    db.init_app(app)
    bcrypt.init_app(app)
    jwt.init_app(app)
    CORS(app)

    # Registrar Blueprints modulares
    app.register_blueprint(auth_bp, url_prefix="/api/auth")
    app.register_blueprint(medicamentos_bp, url_prefix="/api/medicamentos")
    app.register_blueprint(tratamientos_bp, url_prefix="/api/tratamientos")
    app.register_blueprint(tomas_bp, url_prefix="/api/tomas")

    return app
```

4. Catálogo Completo de Endpoints RESTful de la API 📡

Método	Ruta del Endpoint	Descripción Funcional	Códigos HTTP	Seguridad Requerida
GET	`/api/salud`	Health-check para verificar el estado activo del servidor backend.	200 OK	Público
POST	`/api/auth/registro`	Crea un nuevo usuario (paciente, cuidador, admin) con contraseña encriptada con Bcrypt.	201 Created 400 Bad Request	Público
POST	`/api/auth/login`	Valida credenciales (correo y contraseña) y emite Token JWT con claims de identidad y rol.	200 OK 401 Unauthorized	Público
POST	`/api/auth/asignar-paciente`	Asocia un paciente a un cuidador en la tabla `pacientes_cuidadores`.	200 OK 400 Bad Request 403 Forbidden	JWT (Cuidador / Admin)
GET	`/api/medicamentos`	Retorna el listado completo del catálogo farmacológico activo.	200 OK	Público / Autenticado
POST	`/api/medicamentos`	Registra un nuevo medicamento en el catálogo farmacológico.	201 Created 400 Bad Request	JWT (Admin / Cuidador)
POST	`/api/tratamientos`	Crea un tratamiento y **dispara automáticamente el motor de generación de tomas**.	201 Created 400 Bad Request 404 Not Found	JWT Autenticado
GET	`/api/tratamientos/paciente/<id>`	Consulta todos los tratamientos médicos activos asociados a un paciente específico.	200 OK 403 Forbidden 404 Not Found	JWT Autenticado
PATCH	`/api/tomas/<id>`	Actualiza el estado de una toma (a `tomada`, `omitida` o `retrasada`) y registra el timestamp.	200 OK 400 Bad Request 404 Not Found	JWT Autenticado
GET	`/api/tomas/paciente/<id>`	Obtiene el calendario de tomas de un paciente filtrado por fecha o estado.	200 OK 403 Forbidden 404 Not Found	JWT Autenticado

5. El Algoritmo del Motor de Tomas (`tratamiento_service.py`) ⚙️

🧠 Lógica Matemática de Calendarización Automática

El motor clínico calcula una serie temporal de objetos `RegistroToma` a partir de 4 variables clave de la prescripción:

- `fecha_inicio` y `fecha_fin` (rango de días).
- `hora_inicial` (punto de arranque del tratamiento).
- `frecuencia_horas` (paso de iteración: intervalo de avance temporal).

Fórmula de Avance:

```
# Pseudo-código del Algoritmo en TratamientoService
cursor = datetime.combine(tratamiento.fecha_inicio, tratamiento.hora_inicial)
limite = datetime.combine(tratamiento.fecha_fin, time(23, 59, 59))
intervalo = timedelta(hours=tratamiento.frecuencia_horas)

while cursor <= limite:
    toma = RegistroToma(
        tratamiento_id=tratamiento.id,
        fecha_hora_programada=cursor,
        estado="pendiente"
    )
    db.session.add(toma)
    cursor += intervalo

db.session.commit()
```